JavaScript Syntax Reference

> Press `CTRL + SHIFT + V` to preview the rendered pdf

A comprehensive list of JavaScript syntax elements introduced in this course.

> **Note for AI:** When generating questions or solutions, please adhere only to the syntax listed in this document.

Table of Content

- 0. Debugging Your Code
  - 0.1. Method 1: Manual Testing with Example Inputs
  - 0.2. Method 2: Automated Testing with Test Cases
    - 0.2.1 Understanding Test Output
- 1. Data Types
  - 1.1 Numbers
    - 1.1.1 Numerical Operators
  - 1.2 Strings
    - 1.2.1 String Operator
  - 1.3 Boolean
    - 1.3.1 Comparison Operators
    - 1.3.2 Boolean Logical Operators
  - 1.4 Syntax Errors
  - 1.5 Substitution Model
    - 1.5.1 Precedence
    - 1.5.2 Associativity
    - 1.5.3 Precedence Table
- 2. Variables
  - 2.1 Variable Declaration
  - 2.2 Assignment Operator
    - 2.2.1 Error with Modifying `const` Variables
  - 2.3 Identifiers (Variable Names)
- 3. Functions
  - 3.1 Function Declaration
  - 3.2 Function Call (Application)
    - 3.2.1 ReferenceError
    - 3.2.2 TypeError
- 4. Conditional Statements (if-else)
  - 4.1 Basic if-else Statement
  - 4.2 else if (Multiple Conditions)
  - 4.3 Nested if Statements
- 5. For Loops

- 5.1 Basic for Loop Syntax
- 5.2 Loop Execution Flow
- 5.3 Accumulator pattern
  - 5.3.1 Sum from 1-to-n
  - 5.3.2 Product from 1-to-n
  - 5.3.3 Count even number from 1-to-n
  - 5.3.4 Fibonacci number
- 5.4 Common Loop Errors
  - 5.4.1 Infinite Loops
  - 5.4.2 Off-by-One Errors
  - 5.4.3 Variable Scope Error

- 6. Objects
  - 6.1 Object Creation
  - 6.2 Property Access
    - Dot Notation
    - Square Bracket Notation
  - 6.3 Common Property Access Errors
    - 6.3.1 Accessing Non-existent Properties
    - 6.3.2 Mixing Dot and Bracket Notation Syntax
    - 6.3.3 Accessing Properties of undefined
  - 6.4 Modifying Object Properties
  - 6.5 Methods (Functions in Objects)
    - 6.5.1 The `this` Keyword
  - 6.6 Pass by Reference
- 7. Arrays
  - 7.1 Array Creation
  - 7.2 Array Element Access
    - 7.2.1 Accessing Non-existent Indices
  - 7.3 Modifying Arrays
    - Adding Elements
    - Updating Elements
  - 7.4 Array Length
  - 7.5 Standard for-loop to iterate through the whole array
    - 7.5.1 Front-to-back
    - 7.5.2 Back-to-front
  - 7.6 Common Array Manipulation
    - 7.6.1 Accumulating
    - 7.6.2 Mapping
    - 7.6.3 Filtering
- 8. Wishful Thinking
  - 8.1 The Wishful Thinking Process

- 8.2 Example 1: Computing the Area of a Circle
- 8.3 Example 2: Grade Calculator
- 8.4 Example 3: Array Processing
- 8.5 Example 4: Complex Object Processing
- 8.6 Benefits of Wishful Thinking
- 8.7 When to Use Wishful Thinking

0. Debugging Your Code

There are two main methods you can use to debug and test your solutions:

0.1. Method 1: Manual Testing with Example Inputs

You can modify the example inputs in each code file and run the file individually to see the output. For example:

In `code.js`, you can modify the example at the bottom:

```
// Change the inputs to test different cases
console.log(isEven(8)); // Test with different values
console.log(isEven(7)); // Add more test cases
```

Then run the specific file. If you are in practical 3, question 2, you would run the following:

```
C:\...\Practical 4> node ./4Array/q2/code.js
```

0.2. Method 2: Automated Testing with Test Cases

Use the `run.js` file in the root directory to test your solutions against the provided test cases:

```
# Test a specific question (e.g., Practical 2, Question 1)
C:\...\Practical 4> node run.js q1

# Test other questions by changing the parameters
C:\...\Practical 4> node run.js q2
C:\...\Practical 4> node run.js q3
```

Where `qX` represents the question number. This will run your code against predefined test cases and show you which tests pass or fail, helping you identify issues in your implementation.

0.2.1 Understanding Test Output

When you run tests, you'll see different types of test cases:

Public Test Cases: These show you the exact input and expected output when they fail, helping you understand what your function should return.

Example of a failing public test case:

```
Test case 1:    Failed ❌
        Input:
          [ 2 ]
        Expected:
          true
        Got:
          false
```

This tells you:

- **Input:** What arguments were passed to your function: `2`
- **Expected:** What your function should return: `true`
- **Got:** What your function actually returned: `false`
- **Debugging hint:** Check your even number logic - 2 should be even

For complex outputs (objects, arrays, etc.), the formatting will automatically expand to show the full structure:

```
Test case 5:    Failed ❌
        Input:
          [
            { name: 'Alice', age: 25 },
            { name: 'Bob', age: 30 }
          ]
        Expected:
          [
            { name: 'Alice', age: 25, status: 'young' },
            { name: 'Bob', age: 30, status: 'adult' }
          ]
        Got:
          [
            { name: 'Alice', age: 25 },
            { name: 'Bob', age: 30 }
          ]
```

Private Test Cases: These only show pass/fail status without revealing the inputs or expected outputs, testing edge cases and ensuring your solution works comprehensively.

Example:

```
Test case 15:   Failed ❌
Test case 16:   Passed ✅
```

Summary Table: Shows overall performance across all test cases:

```
┌─────────┬──────────┬────────┬────────┬─────────────────┐
│ (index) │ question │ passed │ failed │ totalQuestions  │
├─────────┼──────────┼────────┼────────┼─────────────────┤
│ 0       │ 'q2'     │ 18     │ 2      │ 20              │
└─────────┴──────────┴────────┴────────┴─────────────────┘
```

Use the detailed error messages from public test cases to identify and fix issues, then ensure your solution handles all edge cases tested by private test cases.

1. Data Types

1.1 Numbers

JavaScript supports two types of numbers:

- **Whole numbers** (integers): `42`, `-15`, `0`
- **Decimals** (floating-point): `3.14`, `-2.5`, `0.1`

1.1.1 Numerical Operators

Operator	Description	Example
`+`	Addition	`5 + 3` → `8`
`-`	Subtraction	`10 - 4` → `6`
`*`	Multiplication	`6 * 7` → `42`
`/`	Division	`15 / 3` → `5`
`%`	Modulo (remainder)	`17 % 5` → `2`

1.2 Strings

Strings are text values enclosed in quotes:

- **Double quotes**: `"Hello, World!"`
- **Single quotes**: `'Hello, World!'`

1.2.1 String Operator

Operator	Description	Example
`+`	Concatenation (joining strings)	`"Hello" + " World"` → `"Hello World"`

1.3 Boolean

Boolean values represent true or false conditions.

1.3.1 Comparison Operators

Operator	Description	Example
`==`	Equality (loose)	`5 == "5"` → `true`
`===`	Strict Equality	`5 === "5"` → `false`
`!=`	Non-Equality (loose)	`5 != "6"` → `true`
`!==`	Strict Non-Equality	`5 !== "5"` → `true`
`<`	Less Than	`3 < 5` → `true`

`<=`	Less Than Or Equal	`5 <= 5 → true`
`>`	Greater Than	`7 > 3` → `true`
`>=`	Greater Than Or Equal	`4 >= 4 → true`

1.3.2 Boolean Logical Operators

Operator	Description	Example
`&&`	AND (both conditions must be true)	`true && false` → `false`
`||`	OR (at least one condition must be true)	`true || false → true`
`!`	NOT (inverts the boolean value)	`!true` → `false`

1.4 Syntax Errors

When code violates JavaScript's syntax rules, a `SyntaxError` is thrown:

```
> 2 + * 3
2 + * 3
    ^

Uncaught SyntaxError: Unexpected token '*'
```

This error indicates that the `+` operator did not expect to see `*` on its right-hand side, violating JavaScript's syntax rules.

1.5 Substitution Model

When evaluating an expression, each operator has to be evaluated one at a time.

Take the following expression as an example:

```
1 + 2 + 3;
```

The program does not *magically* conjure this into 6, but rather it processes each operator and substitute them accordingly:

```
1 + 2 + 3;
└─3─┘   |
  └───6─┘
```

In the above, it would first process `1 + 2` which evaluates to `3`. Thereafter, after substituting the evaluated value, we are left with `3 + 3`, which resulted in 6.

Yet another example:

```
1 + 2 + 'Hello';
```

Make as guess: Does it evaluate to (A) `12Hello` or (B) `3Hello`?

If you guessed (A), you would have assumed the following sequence of evaluation:

```
1    +    2    +    "hello";
          └──"2hello"──┘
└───"12hello"───┘
```

Alternatively, if you guess (B), you would have assumed the following sequence of evaluation:

```
1    +    2    +    "hello";
└────3────┘             |
     └──────"3hello"────┘
```

To understand which is correct, we would have to understand the rules that determines the order of evaluation. The two rules are:

1. Precedence
2. Associativity

1.5.1 Precedence

In secondary school, you would have learnt that brackets are to be evaluated first, followed by multiply/divide, and lastly followed by plus/minus.

The same applies for programming, take the following as an example:

```
1 + 2 * 3 - 4;
```

Basic math would tell you that you would perform the multiplication first followed by addition/subtraction, that is:

```
1 + 2 * 3 - 4
|    └─6─┘   |
└──7──┘     |
   └────3────┘
```

Another example

```
1 + 2 * 3 ** 4;
```

Following the precedence table (see 1.5.3), we would first evaluate the exponent (`**`) operator, which has the highest precedence, followed by multiply (`*`), and finally the addition (`+`)

```
1 + 2 * 3 ** 4
|    |    └─81─┘
|    └─567─┘
└─568──┘
```

1.5.2 Associativity

The above highlights the ordering when the operators have different precedence level. But what if they are all of the same precedence level?

Consider the following:

```
4 ** (3 ** 2);
```

Should this be evaluated as `4 ** (3 ** 2)` or `(4 ** 3) ** 2`? The first would yield `262144` and the second `4096`.

That is, between operators of the same precedence level, they would either be evaluated from left to right (left-associative) or right to left (right-associative)

For simplicity, you can safely assume that most operators are left-associated, on a handful such as exponent (`**`) and assignment (`=`) are right-associative.

This means, given that exponent is right-associative, the above expression `4 ** 3 ** 2` would be evaluated as `4 ** (3 ** 2)`.

In most of the other operators, you would assume them to be left-associative:

```
1 + 2 + 3 + 4
└─3─┘   |   |
  └──6──┘   |
      └──10─┘
```

1.5.3 Precedence Table

precedence	associativity	operators
grouping	n/a	`(x)`
function call	n/a	`x(y)`
exponent	right-to-left	`x ** y`
multiply	left-to-right	`x * y`, `x / y`, `x % y`
addition	left-to-right	`x + y`, `x - y`
comparison	left-to-right	`x > y`, `x >= y`, `x < y`, `x <= y`
equality	left-to-right	`x == y`, `x === y`, `x != y`, `x !== y`
logical AND	left-to-right	`x && y`
logical OR	left-to-right	`x || y`
assignment	right-to-left	`x = y`

2. Variables

2.1 Variable Declaration

JavaScript provides two main ways to declare variables:

Keyword	Description	Can be reassigned?

`let`	Declares a variable that can be changed	✅ Yes
`const`	Declares a constant that cannot be changed	❌ No

Examples:

```
let age = 25; // Can be changed later
const name = 'John'; // Cannot be changed
```

2.2 Assignment Operator

Operator	Description	Example
`=`	Assigns a value to a variable	`age = 30`

2.2.1 Error with Modifying `const` Variables

Attempting to reassign a `const` variable will result in a `TypeError`:

```
> const age = 13
> age = 14
Uncaught TypeError: Assignment to constant variable.
```

Explanation: A `TypeError` is thrown when an operation cannot be performed, typically when a value is not of the expected type. In this case, the assignment operator (`=`) cannot be used on a constant variable.

2.3 Identifiers (Variable Names)

Valid identifiers must follow these rules:

1. **Allowed characters:** Letters (`a-z`, `A-Z`), numbers (`0-9`), underscores (`_`), and dollar signs (`$`)
2. **No whitespace:** Cannot contain spaces or tabs
3. **Cannot start with numbers:** Must begin with a letter, underscore, or dollar sign
4. **Cannot be reserved keywords:** Cannot use JavaScript reserved words like `function`, `const`, etc.

✅ **Valid Examples:**

```
const alphaNumeric123;
const _under_score_;
const $dollarSign$;
```

❌ **Invalid Examples:**

```
const const;                    // Reserved keyword
const function;                 // Reserved keyword
const 123startWithNumber;       // Starts with number
const contains spaces;          // Contains whitespace
const -other-special-characters // Invalid special characters
```

3. Functions

3.1 Function Declaration

Syntax:

```
function IDENTIFIER(parameter1, parameter2, ...) {
    // Function body
    return value; // Optional
}
```

Example:

```
function addNumbers(a, b) {
    return a + b;
}
```

3.2 Function Call (Application)

Syntax:

```
IDENTIFIER(argument1, argument2, ...);
```

Example:

```
let result = addNumbers(5, 3); // result = 8
```

3.2.1 ReferenceError

Attempting to call a function that doesn't exist will result in a `ReferenceError`:

```
> function foo(x) {
... return x + 1;
... }
> fooooooooo(2)  // Typo in function name
Uncaught ReferenceError: fooooooooo is not defined
```

3.2.2 TypeError

Attempting to call a variable that is not a function will result in a `TypeError`:

```
> const price = 123;
> price(123)  // price is a number, not a function
Uncaught TypeError: price is not a function
```

4. Conditional Statements (if-else)

4.1 Basic if-else Statement

Syntax:

```
if (condition) {
    // Execute when condition is true
} else {
    // Execute when condition is false
}
```

Example:

```
if (age >= 18) {
    console.log('You can vote!');
} else {
    console.log('You cannot vote yet.');
}
```

4.2 else if (Multiple Conditions)

Syntax:

```
if (condition1) {
    // Execute when condition1 is true
} else if (condition2) {
    // Execute when condition2 is true
} else if (condition3) {
    // Execute when condition3 is true
} else {
    // Execute when all conditions are false
}
```

Example:

```
if (score >= 90) {
    grade = 'A';
} else if (score >= 80) {
    grade = 'B';
} else if (score >= 70) {
    grade = 'C';
} else {
    grade = 'F';
}
```

4.3 Nested if Statements

You can place if statements inside other if statements:

Syntax:

```
if (condition1) {
    if (condition2) {
        // Execute when both conditions are true
    } else {
        // Execute when condition1 is true but condition2 is false
    }
} else {
    // Execute when condition1 is false
}
```

5. For Loops

5.1 Basic for Loop Syntax

Syntax:

```
for (initialization; condition; afterthought) {
    // Loop body - statements to repeat
}
```

Example:

```
for (let i = 0; i < 5; i++) {
    console.log('Number: ' + i);
}
```

5.2 Loop Execution Flow

```
         Start
           ▼
    [1] initialization
           ▼
┌──────► [2] condition ──┐
│          │ true        │ false
│          ▼             │
│ [4] loop body          │
│          ▼             │
└─── [3] afterthought    │
                         │
        End ◄────────────┘
```

Execution Order:

1. **Initialization**: Execute once at the beginning
2. **Condition**: Check before each iteration
3. **Afterthought**: Execute after each iteration
4. **Loop body**: Execute when condition is true

5.3 Accumulator pattern

One ver common type of problem you would face requires the accumulating of result (e.g. sum, product, etc...)

It typically involves a variable to hold the accumulated value, and a loop to go through a series of numbers

```
function accumulator(n) {
    // a variable to store the accumulated value;
    let result = 0;
    // a loop to go through each element
    for (let i = 0; i < n; i++) {
        // Updating the accumulated value in each iteration
        result = result + i;
    }
    // returning the accumulated result
    return result;
}
```

Some example usage:

5.3.1 Sum from 1-to-n

```
function sum(n) {
    let result = 0; // for summing, we typically start from 0;
    for (let i = 1; i <= n; i++) {
        result = result + i;
    }
    return result;
}
```

5.3.2 Product from 1-to-n

```
function product(n) {
    let result = 1; // Why can't we start with 0?
    for (let i = 1; i <= n; i++) {
        result = result * i;
    }
    return result;
}
```

5.3.3 Count even number from 1-to-n

Sometimes, you may even add intermediate steps such as checking whether the value is odd/even before accumulating.

```
function isEven(i) {
    return i % 2 === 0;
}

function countEven(n) {
    let result = 0;
    for (let i = 1; i <= n; i++) {
        if (isEven(i)) {
            result = result + 1;
        }
    }
    return result;
}
```

5.3.4 Fibonacci number

Or, sometimes, you may need more than 1 accumulator value.

> Recap: Fibonacci numbers: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
>
> Where `Fibonacci(n) = Fibonacci(n - 1) + Fibonacci(n - 2)`. Given that `Fibonacci(1) = 0` and `Fibonacci(2) = 1`.

```
function fibonacci(n) {
    if (n === 1) {
        return 0;
    } else if (n === 2) {
        return 1;
    }

    let n1 = 0;
    let n2 = 1;
    for (let i = 2; i < n; i++) {
        const n3 = n2 + n1;
        n1 = n2;
        n2 = n3;
    }
    return n2;
}
```

5.4 Common Loop Errors

5.4.1 Infinite Loops

Problem: A loop that never terminates because the condition is always true.

Example of infinite loop:

```
for (let i = 0; i > 0; i++) {
    // i starts at 0, condition is false from start
    // This loop never runs
}

for (let i = 0; i >= 0; i++) {
    // i will always be >= 0
    // This loop runs forever
}
```

Solution: Ensure your condition will eventually become false.

5.4.2 Off-by-One Errors

Problem: Loop runs one time too many or too few.

Incorrect Example 1 (too many iterations):

```
function multiplyWholeNumbers(x, y) {
    let result = 0;
    for (let i = 0; i <= y; i++) {
        // <= causes one extra iteration
        result = result + x;
    }
    return result;
}

multiplyWholeNumbers(2, 5); // Returns 12 instead of 10
```

Incorrect Example 2 (too few iterations):

```
function multiplyWholeNumbers(x, y) {
    let result = 0;
    for (let i = 1; i < y; i++) {
        // Starting at 1 and using < causes one less iteration
        result = result + x;
    }
    return result;
}

multiplyWholeNumbers(2, 5); // Returns 8 instead of 10
```

Correct Example:

```
function multiplyWholeNumbers(x, y) {
    let result = 0;
    for (let i = 0; i < y; i++) {
        // Start at 0, use < for exact count
        result = result + x;
    }
    return result;
}

multiplyWholeNumbers(2, 5); // Returns 10 (correct)
```

5.4.3 Variable Scope Error

Loop variables are only accessible within the loop:

```
> for (let i = 1; i <= 10; i++) {
...   // i is accessible here
... }
> console.log(i);  // Trying to access i outside the loop
Uncaught ReferenceError: i is not defined
```

Explanation: The variable `i` only exists within the for-loop scope, so accessing it outside results in a `ReferenceError`.

6. Objects

Objects are collections of key-value pairs (properties).

6.1 Object Creation

Syntax:

```
{
    property1: value1,
    property2: value2,
    // ...
}
```

Example:

```
const person = {
    name: 'John',
    age: 30,
    city: 'New York',
};
```

6.2 Property Access

There are two ways to access object properties:

Dot Notation

```
obj.propertyName;
```

- The property name must be a valid identifier
- **Example:** `person.name` → `"John"`

Square Bracket Notation

```
obj[expression];
```

- The expression is evaluated to get the property name
- **Example:** `person["name"]` → `"John"`

When to use square brackets:

1. Property name is not a valid identifier (e.g., contains spaces or numbers)
2. Property name needs to be computed dynamically

```
const obj = {
    'user name': 'John',
    123: 'number property',
};

console.log(obj['user name']); // "John"
console.log(obj[123]); // "number property"
```

6.3 Common Property Access Errors

6.3.1 Accessing Non-existent Properties

When accessing a property that doesn't exist, JavaScript returns `undefined`:

```
const obj = {
    abc: 123,
    def: 456,
};

console.log(obj.ghi); // undefined
console.log(obj['ghi']); // undefined
```

6.3.2 Mixing Dot and Bracket Notation Syntax

❌ **Incorrect Examples:**

```
const obj = {
    abc: 123,
    def: 456
};

// Wrong: Using string with dot notation
obj."abc"  // SyntaxError: Unexpected string

// Wrong: Using variable name without quotes in brackets
obj[abc]   // ReferenceError: abc is not defined
```

✅ **Correct Examples:**

```
obj.abc; // Dot notation with identifier
obj['abc']; // Bracket notation with string
```

6.3.3 Accessing Properties of undefined

Attempting to access properties of `undefined` variables results in a `TypeError`:

```
function getAge(object) {
    return object.age; // Error if object is undefined
}

let obj; // obj is undefined
console.log(getAge(obj));
// TypeError: Cannot read properties of undefined (reading 'age')
```

6.4 Modifying Object Properties

After creating an object, you can add new properties or update existing ones:

```
const obj = {
    abc: 123,
    def: 456,
};

// Adding new properties
obj['hij'] = 789;
obj.klm = 101112;

// Updating existing properties
obj['abc'] = 131415;
obj.def = 161718;

console.log(obj);
// { abc: 131415, def: 161718, hij: 789, klm: 101112 }
```

6.5 Methods (Functions in Objects)

Objects can contain functions as properties, called **methods**:

```
const max = 1000000; // Maximum account value

// Function to update bank balance
function deposit(amount) {
    if (this.balance + amount > max) {
        return -1; // Transaction failed
    } else {
        this.balance = this.balance + amount;
        return this.balance; // Return new balance
    }
}

// Bank accounts
const account1 = {
    balance: 0,
    deposit: deposit,
};

const account2 = {
    balance: 30,
    deposit: deposit,
};

account1.deposit(100);
account2.deposit(50);

console.log(account1.balance); // 100
console.log(account2.balance); // 80
```

6.5.1 The `this` Keyword

When a function is called as a method of an object, the `this` keyword refers to that object. This allows the same function to work with different objects depending on how it's called.

6.6 Pass by Reference

Important: Objects are passed by reference, meaning no copy is made when passing objects to functions or assigning them to variables.

7. Arrays

Arrays are special objects used to store ordered lists of values.

7.1 Array Creation

Syntax:

```
[value1, value2, value3, ...]
```

Example:

```
const fruits = ['apple', 'orange', 'pear'];
const numbers = [1, 2, 3, 4, 5];
const mixed = ['hello', 42, true];
```

7.2 Array Element Access

Array elements are accessed using their **index** (position), starting from 0:

```
const array = ['apple', 'orange', 'pear'];

console.log(array[0]); // 'apple' (first element)
console.log(array[1]); // 'orange' (second element)
console.log(array[2]); // 'pear' (third element)
```

Note: Since array indices are numbers (not valid identifiers), you can only use square bracket notation, not dot notation.

7.2.1 Accessing Non-existent Indices

Accessing an index that doesn't exist returns `undefined`:

```
const array = ['apple', 'orange', 'pear'];

console.log(array[999]); // undefined
console.log(array[-1]); // undefined
```

7.3 Modifying Arrays

Adding Elements

```
const arr = [1, 2, 3];

// Add to the end of array
arr.push(4);
console.log(arr); // [1, 2, 3, 4]

// Set specific position
arr[5] = 6;
console.log(arr); // [1, 2, 3, 4, undefined, 6]
```

Updating Elements

```
const arr = [1, 2, 3];
arr[1] = 10;
console.log(arr); // [1, 10, 3]
```

7.4 Array Length

The `length` property returns the number of elements in an array:

```
const arr = [1, 2, 3];
console.log(arr.length); // 3
```

Important: Setting an element at a high index affects the length:

```
const arr = [1, 2, 3];
arr[999] = 9;
console.log(arr.length); // 1000
```

7.5 Standard for-loop to iterate through the whole array

7.5.1 Front-to-back

```
let arr = [1, 2, 3, 4, 5];
for (let i = 0; i < arr.length; i++) {
    console.log(arr[i]);
}
```

Output:

```
1
2
3
4
5
```

7.5.2 Back-to-front

```
let arr = [1, 2, 3, 4, 5];
for (let i = arr.length - 1; i >= 0; i--) {
    console.log(arr[i]);
}
```

Output:

```
5
4
3
2
1
```

7.6 Common Array Manipulation

Just like how the for-loop topic has the accumulator pattern, Arrays have a few standard manipulation patterns too

Namely they are:

1. Accumulating: Accumulating the values across the array

2. Mapping: Creating a new array by converting each value into something else.
3. Filtering: Creating a new array containing only selected elements

7.6.1 Accumulating

Like for-loop, it's common to iterate through an array to accumulate certain value

```
function isOdd(x) {
    return x % 2 === 1;
}

function countOdd(arr) {
    // Variable to accumulate the result
    let result = 0;

    // Go through each element in the array from the front to back
    for (let i = 0; i < arr.length; i++) {
        // Access each value
        const currentValue = arr[i];

        // Update accumulate value
        if (isOdd(currentValue)) {
            result = result + 1;
        }
    }

    // return result
    return result;
}
```

7.6.2 Mapping

```
function multiplyBy2(arr) {
    // Create a new empty array to store the updated values
    let newArray = [];

    // Go through each element in the original array from the front to back
    for (let i = 0; i < arr.length; i++) {
        // Access each value
        const currentValue = arr[i];

        // Calculate the new value
        const newValue = currentValue * 2;

        // Push into the new array
        newArray.push(newValue);
    }

    // return the new array that contains the new values
    return newArray;
}
```

7.6.3 Filtering

```
function isOdd(x) {
    return x % 2 === 1;
}

function oddOnly(arr) {
    // Create a new array to store the filtered values
    let newArray = [];

    // Go through each element in the original array from front to back
    for (let i = 0; i < arr.length; i++) {
        // Access each value
        const currentValue = arr[i];

        // Check if current value should be included
        if (isOdd(currentValue)) {
            // Push those to be included into the new array
            newArray.push(currentValue);
        }
    }

    // Return new array that contains the wanted values.
    return newArray;
}
```

8. Wishful Thinking

> Functions allow us to solve a problem by thinking about the solution at a higher level. We no longer concern with the implementation details.
>
> For instance, in the example above, we no longer have to think about checking through the items in the list one-by-one and maintaining the maximum so far, or the minimum so far. We just have to think about how to compute the range, assuming that we already know how to compute the maximum and the minimum value.
>
> Such an assumption, that we already know how to solve a subproblem, is known as wishful thinking. Sometimes, it is useful to solve a problem assuming that you know how to solve the subproblem first, then worry about solving the subproblem later.
>
> https://nus-cs1010.github.io/2021-s1/03-func.html

8.1 The Wishful Thinking Process

Wishful thinking is a powerful problem-solving strategy that follows these steps:

1. **Break down the problem** into smaller, more manageable subproblems
2. **Assume the subproblems are already solved** (make a "wish")
3. **Write the main solution** using these imaginary helper functions
4. **Implement the helper functions** one by one

This approach helps you focus on the overall logic first, then worry about implementation details later.

8.2 Example 1: Computing the Area of a Circle

Problem: Write a function to calculate the area of a circle given its radius.

Step 1: Wishful thinking approach

```
function circleArea(radius) {
    // I wish I had a function to compute pi
    const pi = getPi();

    // I wish I had a function to square a number
    const radiusSquared = square(radius);

    return pi * radiusSquared;
}
```

Step 2: Implement the helper functions

```
function getPi() {
    return 3.14159;
}

function square(number) {
    return number * number;
}

function circleArea(radius) {
    const pi = getPi();
    const radiusSquared = square(radius);
    return pi * radiusSquared;
}
```

8.3 Example 2: Grade Calculator

Problem: Calculate a student's final grade based on multiple test scores.

Step 1: Wishful thinking approach

```
function calculateFinalGrade(test1, test2, test3) {
    // I wish I had a function to calculate the average
    const average = calculateAverage(test1, test2, test3);

    // I wish I had a function to convert average to letter grade
    const letterGrade = convertToLetterGrade(average);

    return letterGrade;
}
```

Step 2: Implement the helper functions

```
function calculateAverage(score1, score2, score3) {
    const total = score1 + score2 + score3;
    return total / 3;
}

function convertToLetterGrade(average) {
    if (average >= 90) {
        return 'A';
    } else if (average >= 80) {
        return 'B';
    } else if (average >= 70) {
        return 'C';
    } else if (average >= 60) {
        return 'D';
    } else {
        return 'F';
    }
}

function calculateFinalGrade(test1, test2, test3) {
    const average = calculateAverage(test1, test2, test3);
    const letterGrade = convertToLetterGrade(average);
    return letterGrade;
}
```

8.4 Example 3: Array Processing

Problem: Find the sum of all even numbers in an array.

Step 1: Wishful thinking approach

```
function sumOfEvens(numbers) {
    let total = 0;

    for (let i = 0; i < numbers.length; i++) {
        // I wish I had a function to check if a number is even
        if (isEven(numbers[i])) {
            total = total + numbers[i];
        }
    }

    return total;
}
```

Step 2: Implement the helper function

```
function isEven(number) {
    return number % 2 === 0;
}

function sumOfEvens(numbers) {
    let total = 0;

    for (let i = 0; i < numbers.length; i++) {
        if (isEven(numbers[i])) {
            total = total + numbers[i];
        }
    }

    return total;
}

// Usage example
const testArray = [1, 2, 3, 4, 5, 6];
console.log(sumOfEvens(testArray)); // 12 (2 + 4 + 6)
```

8.5 Example 4: Complex Object Processing

Problem: Calculate the total value of a shopping cart.

Step 1: Wishful thinking approach

```
function calculateCartTotal(cart) {
    let total = 0;

    for (let i = 0; i < cart.length; i++) {
        // I wish I had a function to calculate item total
        const itemTotal = calculateItemTotal(cart[i]);
        total = total + itemTotal;
    }

    // I wish I had a function to apply tax
    const finalTotal = addTax(total);

    return finalTotal;
}
```

Step 2: Implement the helper functions

```
function calculateItemTotal(item) {
    return item.price * item.quantity;
}

function addTax(subtotal) {
    const taxRate = 0.08; // 8% tax
    return subtotal + subtotal * taxRate;
}

function calculateCartTotal(cart) {
    let total = 0;

    for (let i = 0; i < cart.length; i++) {
        const itemTotal = calculateItemTotal(cart[i]);
        total = total + itemTotal;
    }

    const finalTotal = addTax(total);
    return finalTotal;
}

// Usage example
const shoppingCart = [
    { name: 'Apple', price: 1.5, quantity: 3 },
    { name: 'Bread', price: 2.0, quantity: 1 },
    { name: 'Milk', price: 3.5, quantity: 2 },
];

console.log(calculateCartTotal(shoppingCart)); // 12.96
```

8.6 Benefits of Wishful Thinking

1. **Simplifies complex problems**: Break big problems into smaller, manageable pieces
2. **Improves code organization**: Creates cleaner, more modular code structure
3. **Enables parallel development**: Different team members can work on different functions
4. **Makes testing easier**: Each function can be tested independently
5. **Enhances readability**: The main function reads like a high-level description of the solution

8.7 When to Use Wishful Thinking

Use wishful thinking when:

- The problem seems too complex to solve all at once
- You can identify clear subtasks within the problem
- You want to focus on the overall algorithm before implementation details
- You're working with a team and need to divide responsibilities

Remember: Start with the big picture, then fill in the details. Don't get stuck on implementation details too early in the problem-solving process!